Penetration Testing Report Template

Student Assessment Template

Student Name	[Insert your name]
Student ID	23249815
Unit	ISEC6005
Assessment Title	Final Exam Part 2 - Report
Target Machine	[Insert target Name/IP]
Date Submitted	[Insert date]

Important: This report must present findings responsibly. Include only authorised testing activity, clear evidence, impact, and remediation. Long command histories should be moved to the appendix.

REMINDER – what to put here (from the boxes you practised): Two flavours you practised: MELOS = SQLite, STRING context (search box, needs a quote ' to break out) – read the secrets and users tables, crack the NTLM hash, SSH in. VESTA = MariaDB, NUMERIC context (ticket_id, no quote needed, port 9999) – use INTO OUTFILE to write a webshell to C:/VestaPortal/www/assets/ and get RCE as vesta\mira. Set [ENTRY-POINT], [DB-ENGINE], [TABLE] and [REMOTE-SERVICE] to match whichever one your exam box looks like.

Report Sections

1. Executive Summary
2. Scope and Rules of Engagement
3. Testing Methodology
4. Summary of Findings
5. Detailed Findings
6. Attack Path Summary
7. Remediation Plan
8. Conclusion
9. Appendix A. Technical Evidence
10. Appendix B. References

1. Executive Summary

During the authorised assessment of the target application ([TARGET-IP], port [PORT]), the [ENTRY-POINT e.g. search box / id parameter] was found to be vulnerable to SQL injection. User input is placed directly into a database query without sanitisation, allowing an unauthenticated attacker to modify the query and read arbitrary data from the [DB-ENGINE e.g. SQLite / MariaDB] database. This was used to read application secrets (the flag) and the user table, from which a password hash was recovered, cracked offline, and reused to authenticate over [REMOTE-SERVICE e.g. SSH 22] and obtain a second, higher-privilege flag. The highest risk is the SQL injection (rated High). The main attack path is: SQL injection → data/credential extraction → offline crack → authenticated access. The primary root cause is the use of dynamically-built queries instead of parameterised statements.

Area	Student Entry	Guidance
Overall Risk	High	Choose one overall rating and justify it briefly.
Main Impact	An unauthenticated attacker can read arbitrary data from the database, including credentials, and in some cases execute operating-system commands.	Describe what could happen to the system or organisation.
Primary Root Cause	Insecure coding – user input is concatenated into a SQL query instead of using parameterised statements.	Example: misconfiguration, weak access control, exposed credentials.
Top Remediation Priority	Rewrite the affected query using parameterised statements.	State what should be fixed first and why.

2. Scope and Rules of Engagement

2.1 Scope

Asset / Host	IP / URL	Service(s)	In Scope?
[TARGET-HOST e.g. Melos/Vesta]	http://[TARGET-IP]:[PORT]/	HTTP ([PORT]) / [DB-ENGINE]	Yes
[TARGET-HOST e.g. Melos/Vesta]	[TARGET-IP]	[REMOTE-SERVICE e.g. SSH 22]	Yes

2.2 Rules of Engagement

- ☐ Testing was conducted only against authorised targets.
- ☐ Testing avoided unnecessary service disruption.
- ☐ Evidence was recorded for each finding.
- ☐ Sensitive data was handled responsibly and not disclosed unnecessarily.
- ☐ No persistence was installed unless explicitly required by the lab instructions.

3. Testing Methodology

Phase	Purpose	Evidence to Include
Reconnaissance	Find the web app and the input that reaches the database.	`nmap -sV [TARGET-IP] (find web app on [PORT])`
Enumeration	Confirm injection with a quote and identify the DB engine.	`' in [ENTRY-POINT] -> DB error (identify [DB-ENGINE]) ; ' ORDER BY N-- -`

Vulnerability Analysis	Find the column count and visible columns for UNION.	`' UNION SELECT 1..N-- - reveals visible columns = injectable`
Exploitation / Validation	Extract data (flag/creds), or write a webshell via OUTFILE.	`UNION SELECT ... FROM [TABLE]-- - (read secrets/flag/hashes) OR INTO OUTFILE webshell`
Privilege Escalation	Crack a recovered hash and reuse it to log into the host.	`hashcat -m [MODE] hash.txt rockyou.txt ; ssh [USER]@[TARGET-IP]`
Remediation Advice	Explain parameterised queries and verify the fix.	`parameterised queries ; least-privilege DB (no FILE) ; generic errors`

4. Summary of Findings

ID	Finding	Risk	Affected Asset	Status
F-01	SQL Injection in the [ENTRY-POINT] parameter	High	[TARGET-IP] ([PORT])	Open
F-02	Weak/Reused Password Recovered from Exposed Hash	Medium	[TARGET-IP]	Open

REMINDER (Summary of Findings): Example for this box: F-01 = SQL Injection in the [ENTRY-POINT] parameter (High). Add F-02 = Weak/Reused Password Recovered from Exposed Hash only if you also reached the intermediate flag; delete the unused F-03–F-05 rows.

5. Detailed Findings

REMINDER (Detailed Findings): Example for this box: Finding 1 = SQL Injection in the [ENTRY-POINT] parameter. Replace the Evidence block with YOUR own command output / screenshots. Keep Finding 2 (Weak/Reused Password Recovered from Exposed Hash) only if you reached the intermediate flag by reusing credentials – otherwise delete it.

Finding 1: SQL Injection in the [ENTRY-POINT] parameter

Risk Rating	High
Affected Asset(s)	http://[TARGET-IP]:[PORT]/ ([ENTRY-POINT] parameter)
Vulnerability Type	Insecure coding – SQL injection
Evidence Reference	E-01, E-02
Business Impact	An attacker can read or modify database data and may gain command execution.
Remediation Priority	Immediate

Description

The [ENTRY-POINT] parameter is concatenated into a SQL query executed by the [DB-ENGINE] backend. Supplying a single quote produced a database error, confirming the input breaks out of the query and identifying the engine. Using UNION-based injection, the number of columns was determined and data was extracted directly onto the page, including application secrets and the user table. The weakness is that the application builds SQL by string concatenation rather than using parameterised queries, so client input reaches the database engine as executable code. (On exam day, change [TARGET-IP], [PORT], [ENTRY-POINT] and [DB-ENGINE], and use that engine's catalogue – e.g. sqlite_schema for SQLite, information_schema for MySQL/MSSQL – the method stays the same.)

Evidence

```
'                                    # DB error -> injectable + identifies the engine
' ORDER BY N-- -                     # increase N to find the column count
' UNION SELECT 1,2,3-- -             # identify which columns are visible
' UNION SELECT <name,sql FROM sqlite_schema | table_name,column_name FROM
information_schema...>-- -
' UNION SELECT <cols> FROM [TABLE]-- -  # extract secrets / flag / password hashes
```

Impact

The injection allows reading any data the database account can reach – application secrets and the user table including password hashes. A recovered hash was cracked and reused to log in over [REMOTE-SERVICE], so a single web flaw led to interactive server access and a higher-privilege flag. Where the database account holds the FILE privilege, the same flaw can write a web shell and achieve operating-system command execution. In business terms this is a data breach with credential theft and potential server takeover.

Recommendation

Rewrite the affected query using parameterised queries / prepared statements so input is treated strictly as data (the primary fix); add server-side input validation as defence-in-depth; run the database account with least privilege and without the FILE privilege; and return generic error messages so the backend and schema are not disclosed.

Verification After Fix

Repeat the quote test and the UNION payloads – they should no longer alter the query or return data, and the database error should no longer be shown. Confirm the entry point returns only legitimate results.

Finding 2: Weak/Reused Password Recovered from Exposed Hash

Risk Rating	Medium
Affected Asset(s)	[TARGET-IP], [REMOTE-SERVICE e.g. SSH 22]
Vulnerability Type	Weak credential / credential reuse
Evidence Reference	E-03
Business Impact	A cracked, reused password gives authenticated access to the host.
Remediation Priority	Short-term

Description

A password hash extracted via the SQL injection matched a password in common wordlists and was cracked in seconds. The same password was valid for [REMOTE-SERVICE], allowing lateral movement from the web application to an interactive shell. The weakness is a weak password combined with reuse across services. (Remove this finding if your box only yields the easy flag.)

Evidence

```
hashid '[HASH]'                          # identify the hash type
hashcat -m [MODE] hash.txt rockyou.txt    # -> [PASSWORD]  (e.g. -m 1000 for NTLM)
ssh [USER]@[TARGET-IP]                    # authenticate with the cracked password
cat /home/[USER]/flag.txt                 # higher-privilege flag
```

Impact

A single disclosed hash became full interactive access because the password was weak and reused, turning an information-disclosure issue into host compromise.

Recommendation

Enforce a strong password policy, prohibit password reuse across services, and store credentials using a slow, salted algorithm. Rotate the exposed credentials.

Verification After Fix

Attempt to crack the new hash with the same wordlist – it should no longer fall; confirm the old password is rejected.

6. Attack Path Summary

Step	Evidence	Security Meaning
Initial access / foothold	' in [ENTRY-POINT]	A quote caused a database error, confirming the input is injectable and revealing the engine.
Local enumeration	' ORDER BY N-- - ; ' UNION SELECT 1,2,3-- -	Determined the column count and which columns are visible.
Credential or configuration weakness	UNION from the DB catalogue + target table	Extracted application secrets (the flag) and password hashes.
Service or application abuse	hashcat -m [MODE] hash.txt rockyou.txt	Cracked a recovered hash to a plaintext password.

Privilege escalation proof	ssh/winrm [USER]@[TARGET-IP]	Reused the password to gain authenticated access and the higher-privilege flag.
Final impact	–	Database data disclosed and interactive host access obtained.

Attack Path Diagram

SQL injection in [ENTRY-POINT] → UNION extraction of secrets + hashes → offline crack → authenticated access via [REMOTE-SERVICE] → host flag read.

7. Remediation Plan

Priority	Finding ID	Recommended Action	Owner	Verification Method
Immediate	F-01	Parameterise the vulnerable query and rotate any exposed credentials.	Development team	Re-test the quote/UNION payloads – they must no longer return data.
Short-term	F-02	Parameterise all remaining queries, add input validation and generic errors, and enforce a strong password policy.	Development team	Confirm injection payloads fail and old passwords are rejected.
Long-term	–	Run the database account with least privilege (no FILE), add SQLi test cases to the pipeline, and consider a WAF.	Security team	Periodic code review and testing.

8. Conclusion

The application was vulnerable to SQL injection because user input was placed directly into a database query. This allowed confidential data to be read and, through a recovered and reused

password, led to interactive server access and a higher-privilege flag. The vulnerability is high-impact but well-understood and is fully remediated by parameterised queries and least-privilege database accounts. All testing stayed within the authorised scope; data was read to demonstrate impact but was not altered or destroyed.

Appendix A. Technical Evidence

Use this appendix for detailed command output, screenshots, and supporting notes. Keep the main report concise.

Evidence ID	Command / Source	Screenshot / Output	Purpose
E-01	' in [ENTRY-POINT] (DB error)	[screenshot]	Confirms the injection point and the database engine.
E-02	' UNION SELECT ... FROM [TABLE]-- -	[screenshot]	Shows data/credentials extracted via the injection.
E-03	hashcat -m [MODE] ... ; ssh [USER]@[TARGET-IP]	[screenshot]	Shows the cracked password and authenticated access.

Appendix B. References

OWASP Web Security Testing Guide – SQL injection testing; OWASP Top 10 A03 Injection; OWASP SQL Injection Prevention Cheat Sheet (parameterised queries).

Student Submission Checklist

- ☐ Executive summary is written for a non-technical reader.
- ☐ Each finding includes risk, evidence, impact, and remediation.
- ☐ Screenshots are labelled and referenced in the text.
- ☐ Only authorised testing activity is included.
- ☐ Sensitive credentials are redacted unless required by assessment instructions.
- ☐ The final report is professional, concise, and free of unnecessary terminal history.